

1. 2D Simulation of a Parallel-Plate Capacitor with Finite Dimensions

Develop a numerical 2D electrostatic simulation to compute the electric potential $V(x, y)$ and the electric field $\mathbf{E}(x, y)$ for a parallel-plate capacitor with finite plate dimensions. Use a discretized square computational domain and solve Laplace's equation iteratively.

- **Computational domain:** A square region of size $5 \text{ mm} \times 5 \text{ mm}$ discretized into an $N \times N$ grid with $N = 120$.
- **Top plate (Plate A):** Fixed potential $V = +5 \text{ V}$, extending from $x = -2.5 \text{ mm}$ to $x = 2.5 \text{ mm}$ at $y = +\frac{5}{3} \text{ mm}$.
- **Bottom plate (Plate B):** Fixed potential $V = -5 \text{ V}$, extending from $x = -2.5 \text{ mm}$ to $x = 2.5 \text{ mm}$ at $y = -\frac{5}{3} \text{ mm}$.
- **Medium:** Assume a uniform dielectric with relative permittivity $\kappa = 1$ (vacuum).
- **Optional extension:** Introduce a dielectric interface:
 - Region $y > 0$: $\kappa_1 = 4$
 - Region $y < 0$: $\kappa_2 = 1$
- **Boundary condition:** All outer boundaries of the computational domain are grounded ($V = 0$).
- **Iteration scheme:** Solve Laplace's equation using the nearest-neighbor averaging method for at least 1000 iterations.

Required outputs

- Plot the electric field lines within the simulation domain.
- Plot the equipotential contour map of $V(x, y)$.
- Plot the variation of the electric field magnitude along the vertical line passing through the midpoint of the two plates.

Solution

(TA-Credit: Yashwant - Parallel plate capacitor solution code)

The following Python script simulates the potential distribution and electric field using the relaxation method on a 120×120 grid.

initial code:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.colors as colors
4
5 # 1. Grid and Simulation Parameters
6 N = 120 # Grid resolution
7 iterations = 1000 # More iterations needed for open boundaries
8 V = np.zeros((N, N))
9
10 # Dielectric Constants
11 K_bottom = 1.0 # Vacuum/Air
12 K_top = 1.0 # Dielectric material
13 mid_y = N // 2
14
15 # Define Plate Dimensions (Finite plates for fringing)
16 plate_left = N // 4
17 plate_right = 3 * N // 4
18 top_plate_y = N // 3
19 bottom_plate_y = 2*N // 3
20
21 # 2. Iterative Solve
22 for _ in range(iterations):
23     # Apply Fixed Boundary Conditions (The Plates)
24     V[top_plate_y, plate_left:plate_right] = -5 # Top Plate (+5V)
25     V[bottom_plate_y, plate_left:plate_right] = 5 # Bottom Plate (-5V)
26
27     # Laplace Update for Internal Points (excluding the plates)
28     # We use a copy to avoid updating points mid-iteration (Jacobi style)
29     V_old = V.copy()
30
31     # Bulk regions update
32     # Note: We iterate over all points except the edges
33     for i in range(1, N-1):
34         for j in range(1, N-1):
35             # Skip the actual plates
36             if i == top_plate_y and plate_left <= j < plate_right: continue
37             if i == bottom_plate_y and plate_left <= j < plate_right: continue
38
39             # Dielectric interface logic at mid_y
40             if i == mid_y:
41                 V[i, j] = (K_top * V_old[i-1, j] + K_bottom * V_old[i+1, j] +
42                             ((K_top + K_bottom)/2) * (V_old[i, j-1] + V_old[i, j+1]))
43                 / (2 * (K_top + K_bottom))
44             else:
45                 V[i, j] = 0.25 * (V_old[i+1, j] + V_old[i-1, j] + V_old[i, j+1] +
46                                     V_old[i, j-1])
47
48 # 3. Calculate Electric Field E = -grad(V)
49 dx = 10e-3 / N # Total range (10mm) divided by number of points
50 Ey, Ex = np.gradient(-V, dx)
51 E_mag = np.sqrt(Ex**2 + Ey**2)

```

```

51 # 4.Plots
52 fig, ax = plt.subplots(figsize=(12, 8))
53
54 # Plot 1: Potential (Background)
55 # We use a lower alpha (transparency) so the arrows pop out
56 im_v = ax.imshow(V, cmap='RdYlBu_r', origin='lower', extent=[-5e-3, 5e-3, -5e-3, 5e-3], alpha=0.5)
57 cbar_v = fig.colorbar(im_v, ax=ax, pad=0.08)
58 cbar_v.set_label('Potential (V)')
59
60 # Plot 2: Electric Field Vectors
61 skip = 4
62 Ex_norm = Ex / (E_mag + 1e-10)
63 Ey_norm = Ey / (E_mag + 1e-10)
64 X_grid, Y_grid = np.meshgrid(np.linspace(-5e-3, 5e-3, N), np.linspace(-5e-3, 5e-3, N))
65
66 # Assign quiver to a variable 'qvr'
67 qvr = ax.quiver(X_grid[::skip, ::skip], Y_grid[::skip, ::skip],
68                Ex_norm[::skip, ::skip], Ey_norm[::skip, ::skip],
69                E_mag[::skip, ::skip], cmap='inferno',
70                norm=colors.LogNorm(vmin=0.1, vmax=E_mag.max()), scale=40)
71
72 # Add the Second Colorbar specifically for the Electric Field
73 cbar_e = fig.colorbar(qvr, ax=ax, fraction=0.046, pad=0.04)
74 cbar_e.set_label('Electric Field Magnitude |E| (V/m)')
75
76 # Draw the plates
77 ax.plot([-2.5e-3, 2.5e-3], [1.7e-3, 1.7e-3], 'r-', lw=4, label='Plate +5V')
78 ax.plot([-2.5e-3, 2.5e-3], [-1.7e-3, -1.7e-3], 'b-', lw=4, label='Plate -5V')
79 ax.axhline(0, color='gray', linestyle='--', alpha=0.5)
80
81 ax.set_title('Capacitor Potential (BG) and Field Vectors (Arrows)')
82 ax.set_xlabel('x')
83 ax.set_ylabel('y')
84 ax.legend(loc='upper right')
85
86 # 2. Extract the magnitude along that vertical slice
87 slice_x_idx = (plate_left+plate_right)//2
88 y_axis = np.linspace(-5e-3, 5e-3, N)
89 E_vertical_slice = E_mag[:, slice_x_idx]
90
91 # 3. Create the Plot
92 plt.figure(figsize=(8, 6))
93 plt.plot(y_axis, E_vertical_slice, color='crimson', linewidth=2, label='Field Magnitude |E|')
94
95 # 4. Annotate the Dielectric Interface
96 # The field magnitude changes because  $D = \epsilon \cdot E$  must be constant.
97 # Higher K means Lower E.
98 plt.axvline(0, color='green', linestyle='--', label='Dielectric Interface (y=0)')
99

```

```

100 # 5. Annotate the Plate Positions
101 plt.axvspan(y_axis[top_plate_y], y_axis[bottom_plate_y], color='yellow', alpha=0.1,
102             label='Inside Capacitor')
103
104 # 6. Formatting
105 plt.title('Field Magnitude vs. Y-Position (Connecting the Plates)')
106 plt.xlabel('y (Position from Top to Bottom)')
107 plt.ylabel('Field Magnitude |E| (V/m)')
108 plt.grid(True, linestyle=':', alpha=0.6)
109 plt.legend()
110 plt.show()

```

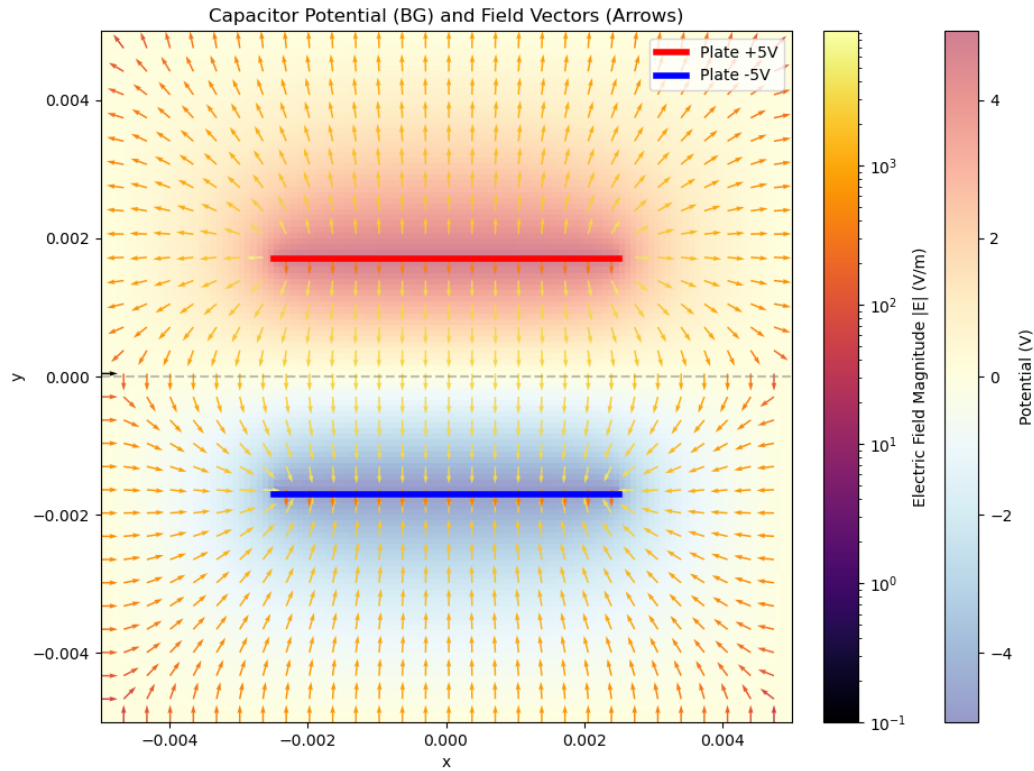


Figure 1: Potential contour plot and the corresponding electric field vectors

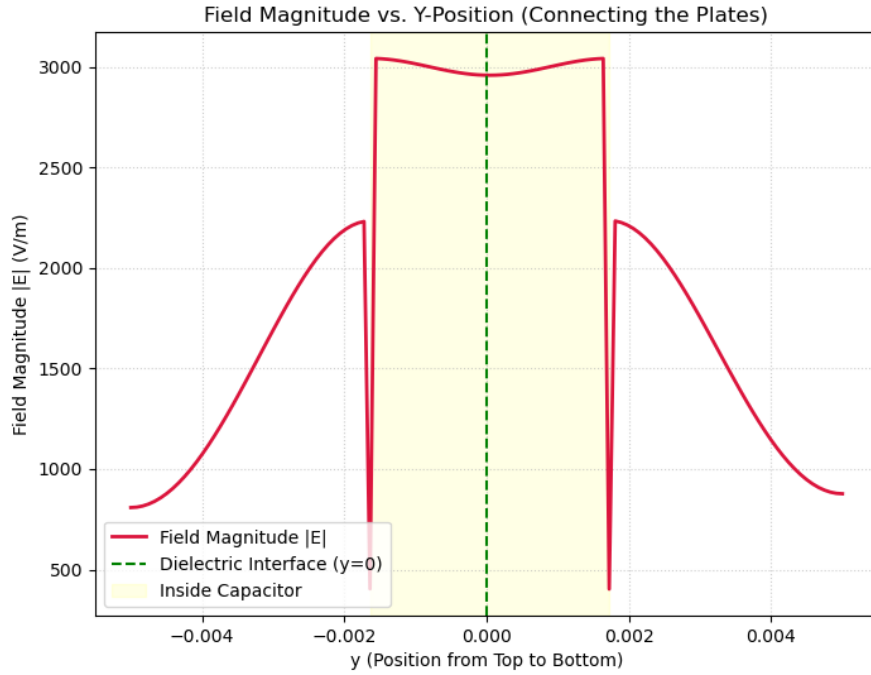


Figure 2: Electric field magnitude along the line joining the midpoint of capacitor plates

Second code:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.colors as colors
4
5 # 1. Grid and Simulation Parameters
6 N = 120 # Grid resolution
7 iterations = 1000 # More iterations needed for open boundaries
8 V = np.zeros((N, N))
9
10 # Dielectric Constants
11 K_bottom = 1.0 # Vacuum/Air
12 K_top = 1.0 # Dielectric material
13 mid_y = N // 2
14
15 # Define Plate Dimensions (Finite plates for fringing)
16 plate_left = 0
17 plate_right = N
18 top_plate_y = N // 6
19 bottom_plate_y = 5*N // 6
20
21 # 2. Iterative Solver (Relaxation)
```

```

22 for _ in range(iterations):
23     # Apply Fixed Boundary Conditions (The Plates)
24     V[top_plate_y, plate_left:plate_right] = -5      # Top Plate (+5V)
25     V[bottom_plate_y, plate_left:plate_right] = 5     # Bottom Plate (-5V)
26
27     # Laplace Update for Internal Points (excluding the plates)
28     # We use a copy to avoid updating points mid-iteration (Jacobi style)
29     V_old = V.copy()
30
31     # Bulk regions update
32     # Note: We iterate over all points except the edges
33     for i in range(1, N-1):
34         for j in range(1, N-1):
35             # Skip the actual plates
36             if i == top_plate_y and plate_left <= j < plate_right: continue
37             if i == bottom_plate_y and plate_left <= j < plate_right: continue
38
39             # Dielectric interface logic at mid_y
40             if i == mid_y:
41                 V[i, j] = (K_top * V_old[i-1, j] + K_bottom * V_old[i+1, j] +
42                           ((K_top + K_bottom)/2) * (V_old[i, j-1] + V_old[i, j+1]))
43                 / (2 * (K_top + K_bottom))
44             else:
45                 V[i, j] = 0.25 * (V_old[i+1, j] + V_old[i-1, j] + V_old[i, j+1] +
46                                   V_old[i, j-1])
47
48 # 3. Calculate Electric Field E = -grad(V)
49 dx = 10e-3 / N # Total range (10mm) divided by number of points
50 Ey, Ex = np.gradient(-V, dx)
51 E_mag = np.sqrt(Ex**2 + Ey**2)
52
53 # 4. Visualization
54 fig, ax = plt.subplots(figsize=(12, 8))
55
56 # Plot 1: Potential (Background)
57 # We use a lower alpha (transparency) so the arrows pop out
58 im_v = ax.imshow(V, cmap='RdYlBu_r', origin='lower', extent=[-2.5e-3, 2.5e-3, -2.5e-3, 2.5e-3], alpha=0.5)
59 cbar_v = fig.colorbar(im_v, ax=ax, pad=0.08)
60 cbar_v.set_label('Potential (V)')
61
62 # Plot 2: Electric Field Vectors (Quiver)
63 skip = 4
64 Ex_norm = Ex / (E_mag + 1e-10)
65 Ey_norm = Ey / (E_mag + 1e-10)
66 X_grid, Y_grid = np.meshgrid(np.linspace(-2.5e-3, 2.5e-3, N), np.linspace(-2.5e-3, 2.5e-3, N))
67
68 # Assign quiver to a variable 'qvr'
69 qvr = ax.quiver(X_grid[::skip, ::skip], Y_grid[::skip, ::skip],
70                 Ex_norm[::skip, ::skip], Ey_norm[::skip, ::skip],
71                 E_mag[::skip, ::skip], cmap='inferno',

```

```

70         norm=colors.LogNorm(vmin=0.1, vmax=E_mag.max()), scale=40)
71
72 # Add the Second Colorbar specifically for the Electric Field
73 cbar_e = fig.colorbar(qvr, ax=ax, fraction=0.046, pad=0.04)
74 cbar_e.set_label('Electric Field Magnitude |E| (V/m)')
75
76 # Draw plates and labels
77 ax.plot([-2.5e-3, 2.5e-3], [1.7e-3, 1.7e-3], 'r-', lw=4, label='Plate +5V')
78 ax.plot([-2.5e-3, 2.5e-3], [-1.7e-3, -1.7e-3], 'b-', lw=4, label='Plate -5V')
79 ax.axhline(0, color='gray', linestyle='--', alpha=0.5)
80
81 ax.set_title('Capacitor Potential (BG) and Field Vectors (Arrows)')
82 ax.set_xlabel('x')
83 ax.set_ylabel('y')
84 ax.legend(loc='upper right')
85 # 2. Extract the magnitude along that vertical slice
86 # Note: we flip the y_axis because in imshow 'lower' origin is used
87 slice_x_idx = (plate_left+plate_right)//2
88
89 y_axis = np.linspace(-2.5e-3, 2.5e-3, N)
90 E_vertical_slice = E_mag[:, slice_x_idx]
91
92 # 3. Create the Plot
93 plt.figure(figsize=(8, 6))
94 plt.plot(y_axis, E_vertical_slice, color='crimson', linewidth=2, label='Field
    Magnitude |E|')
95
96 # 4. Annotate the Dielectric Interface
97 # The field magnitude changes because  $D = \epsilon \cdot E$  must be constant.
98 # Higher  $K$  means Lower  $E$ .
99 plt.axvline(0, color='green', linestyle='--', label='Dielectric Interface (y=0)')
100
101 # 5. Annotate the Plate Positions
102 plt.axvspan(y_axis[top_plate_y], y_axis[bottom_plate_y], color='yellow', alpha=0.1,
    label='Inside Capacitor')
103
104 # 6. Formatting
105 plt.title('Field Magnitude vs. Y-Position (Connecting the Plates)')
106 plt.xlabel('y (Position from Top to Bottom)')
107 plt.ylabel('Field Magnitude |E| (V/m)')
108 plt.grid(True, linestyle=':', alpha=0.6)
109 plt.legend()
110
111 plt.show()

```

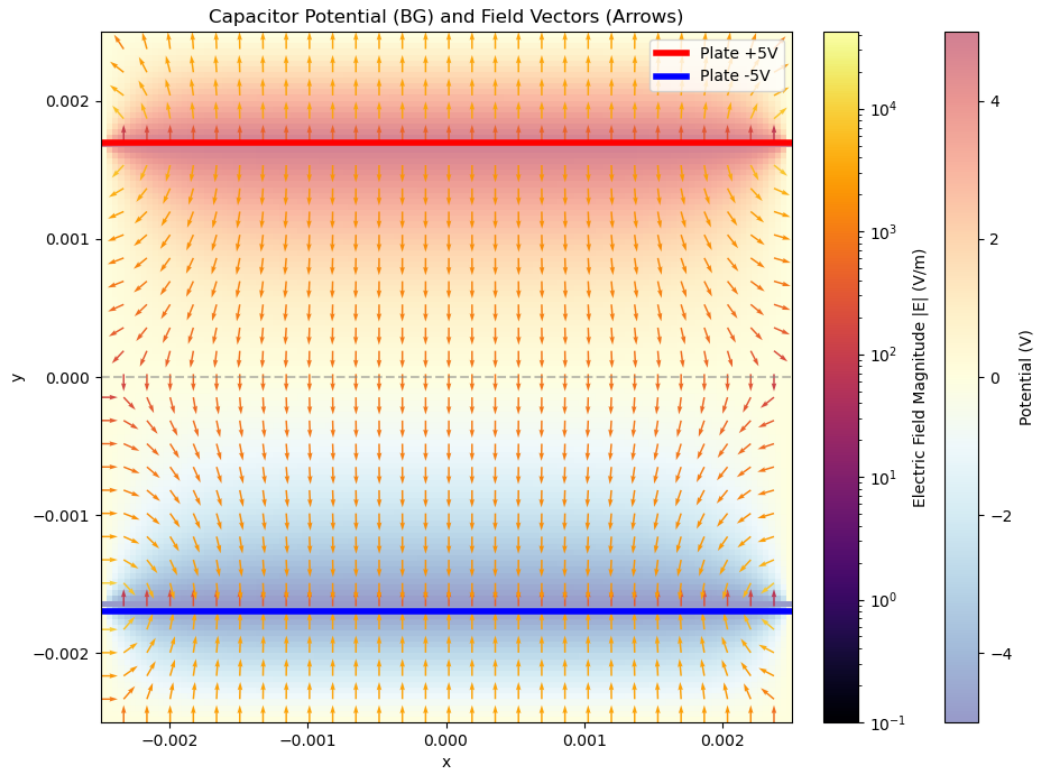


Figure 3: second code's plot

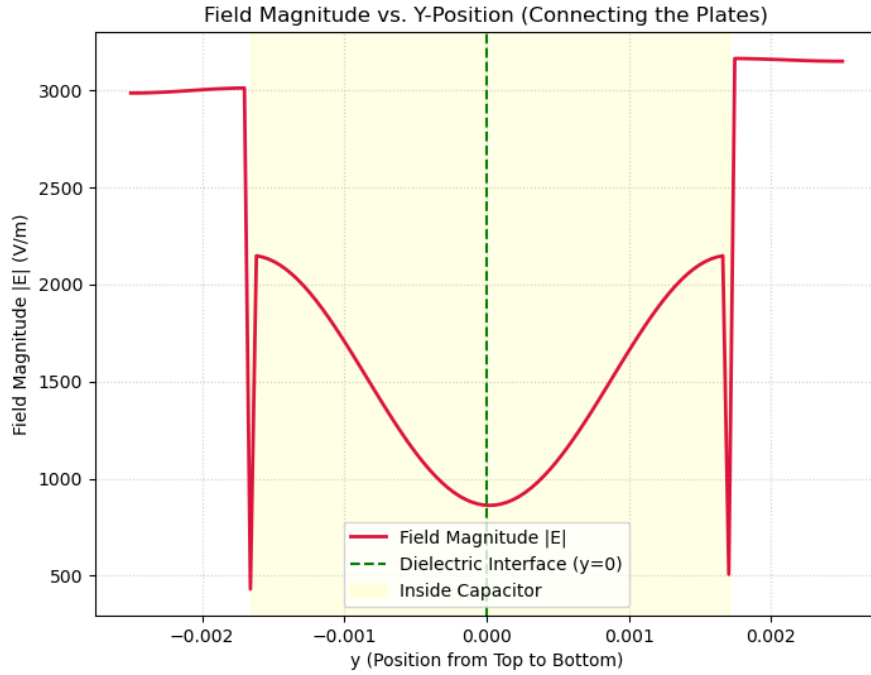


Figure 4: second codes' plot

2. 2D Simulation of a Point Charge Near a Grounded Conducting Sphere

Develop a numerical simulation to study the electrostatic interaction between a point charge and a grounded conducting sphere. Investigate how the induced charge distribution on the sphere changes with the position and magnitude of the external charge.

- **Conducting sphere:** Centered at the origin with radius $R = 2$ (in simulation units) and held at $V = 0$.
- **Point charge:** $Q = 10 \mu\text{C}$ placed at $(x, y) = (4, 0)$.
- **Simulation region:** Square region defined by $-6 \leq x \leq 6$ and $-6 \leq y \leq 6$.
- Perform additional simulations by:
 - Changing the magnitude of the point charge.
 - Changing the distance of the point charge from the sphere.

Required outputs

- Plot the induced surface charge density on the sphere.
- Plot the equipotential contours in the simulation region.

- Plot the electric field vector distribution in the simulation region.

Solution

TA Credit: Sidharth - Python code for Grounded conducting Sphere

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use('TkAgg')
4 import matplotlib.pyplot as plt
5 from matplotlib.widgets import Slider, RadioButtons, Button, TextBox
6 from matplotlib.patches import Circle
7 from matplotlib.collections import LineCollection
8 from matplotlib.colors import Normalize, LogNorm
9 import matplotlib.cm as cm
10
11 k = 8.99e9
12 q_ext = 10e-6
13 limit = 6
14 n_grid = 150
15 R_curr = 2.0
16
17 def get_physics_data_2d(R, a, q):
18     q_img = -q * (R / a)
19     a_img = (R**2) / a
20
21     theta = np.linspace(0, 2*np.pi, 200)
22     xs, ys = R*np.cos(theta), R*np.sin(theta)
23
24     x = np.linspace(-limit, limit, n_grid)
25     y = np.linspace(-limit, limit, n_grid)
26     X, Y = np.meshgrid(x, y, indexing='ij')
27
28     r1 = np.sqrt((X - a)**2 + Y**2) + 0.05
29     r2 = np.sqrt((X - a_img)**2 + Y**2) + 0.05
30     pot = k * (q_ext/r1 + q_img/r2)
31     pot[np.sqrt(X**2 + Y**2) < R] = 0
32
33     Ex = -np.gradient(pot, x, y, axis=(0, 1))[0]
34     Ey = -np.gradient(pot, x, y, axis=(0, 1))[1]
35     emag = np.sqrt(Ex**2 + Ey**2)
36
37     r_surf = np.sqrt((xs - a)**2 + ys**2)
38     sigma = -(q / (4 * np.pi * R)) * (a**2 - R**2) / (r_surf**3)
39
40     return xs, ys, X, Y, pot, Ex, Ey, emag, sigma
41
42 def create_plot(a_val, q_val, mode):
43     fig_clean, ax_clean = plt.subplots(figsize=(7, 6))
44     plt.subplots_adjust(left=0.12, right=0.85)

```

```

45  xs, ys, X, Y, pot, Ex, Ey, emag, sigma = get_physics_data_2d(R_curr, a_val,
46  q_val)
47
48  cbar_ax = fig_clean.add_axes([0.88, 0.15, 0.03, 0.7])
49
50  if mode == "Charge Mode":
51      vmin, vmax = np.min(sigma), np.max(sigma)
52
53      points = np.array([xs, ys]).T.reshape(-1, 1, 2)
54      segments = np.concatenate([points[:-1], points[1:]], axis=1)
55      lc = LineCollection(segments, cmap='viridis', norm=Normalize(vmin=vmin, vmax=
56      vmax))
57      lc.set_array(sigma)
58      lc.set_linewidth(2)
59      ax_clean.add_collection(lc)
60
61      sm = plt.cm.ScalarMappable(cmap='viridis', norm=Normalize(vmin=vmin, vmax=
62      vmax))
63      sm.set_array([])
64      cbar = plt.colorbar(sm, cax=cbar_ax)
65      cbar.set_label(r'$\sigma$ (C/m$^2$)')
66
67      ax_clean.set_title(r"Surface Charge Density ($a=" + f"{a_val:.2f}$)")
68
69  elif mode == "Potential Mode":
70      pot_clim = (100, 50000)
71      levels = np.linspace(pot_clim[0], pot_clim[1], 25)
72
73      pot_plot = pot.copy()
74      inside = np.sqrt(X**2 + Y**2) < R_curr
75      pot_plot[inside] = np.nan
76
77      contour = ax_clean.contourf(X, Y, pot_plot, levels=levels, cmap='Reds')
78      ax_clean.contour(X, Y, pot_plot, levels=levels, colors='darkred', linewidths
79      =0.5)
80
81      cbar = fig_clean.colorbar(contour, cax=cbar_ax)
82      cbar.set_label(r'Potential (V)')
83
84      circle = Circle((0, 0), R_curr, fill=True, facecolor='white', edgecolor='
85      black', linewidth=2)
86      ax_clean.add_patch(circle)
87      ax_clean.set_title(r"Potential Field ($a=" + f"{a_val:.2f}$)")
88
89  elif mode == "Field Mode":
90      skip = 6
91
92      Xs = X[::skip, ::skip]
93      Ys = Y[::skip, ::skip]
94      Exs = Ex[::skip, ::skip]
95      Eys = Ey[::skip, ::skip]

```



```

92     emag_s = emag[:, :skip, ::skip]
93
94     dist_from_sphere = np.sqrt(Xs**2 + Ys**2)
95     dist_from_charge = np.sqrt((Xs-a_val)**2 + Ys**2)
96     mask = (dist_from_sphere > R_curr + 0.2) & (dist_from_charge > 0.3)
97
98     Xm = Xs[mask]
99     Ym = Ys[mask]
100     Exm = Exs[mask]
101     Eym = Eys[mask]
102     emag_m = emag_s[mask]
103
104     emag_nonzero = emag_m[emag_m > 0]
105     if len(emag_nonzero) > 0:
106         emag_min = np.min(emag_nonzero)
107         emag_max = np.max(emag_nonzero)
108         use_log = True
109     else:
110         emag_min, emag_max = 1, 100
111         use_log = False
112
113     lens = np.sqrt(Exm**2 + Eym**2)
114     Ex_unit = np.zeros_like(Exm)
115     Ey_unit = np.zeros_like(Eym)
116     nonzero = lens > 0
117     Ex_unit[nonzero] = Exm[nonzero] / lens[nonzero]
118     Ey_unit[nonzero] = Eym[nonzero] / lens[nonzero]
119
120     if use_log:
121         quiver = ax_clean.quiver(Xm, Ym, Ex_unit * 1.5, Ey_unit * 1.5,
122                                 emag_m, cmap='plasma',
123                                 norm=LogNorm(vmin=emag_min, vmax=emag_max),
124                                 pivot='mid', width=0.003, headwidth=3,
125                                 headlength=3, headaxislength=3)
126         sm = plt.cm.ScalarMappable(cmap='plasma', norm=LogNorm(vmin=emag_min,
127                                                                 vmax=emag_max))
128     else:
129         quiver = ax_clean.quiver(Xm, Ym, Ex_unit * 1.5, Ey_unit * 1.5,
130                                 emag_m, cmap='plasma',
131                                 pivot='mid', width=0.003, headwidth=3,
132                                 headlength=3, headaxislength=3)
133         sm = plt.cm.ScalarMappable(cmap='plasma')
134
135     sm.set_array([])
136     cbar = plt.colorbar(sm, cax=cbar_ax)
137     cbar.set_label(r'$|E|$ (V/m)')
138
139     circle = Circle((0, 0), R_curr, fill=False, color='white', linewidth=2)
140     ax_clean.add_patch(circle)
141     ax_clean.set_title(r"Electric Field ($a=" + f"{a_val:.2f}$)")
142
143     circle = Circle((0, 0), R_curr, fill=False, color='cornflowerblue', linewidth=2)

```

```

141     ax_clean.add_patch(circle)
142     ax_clean.plot(a_val, 0, 'r.', markersize=8, label=r'Source Charge ($q=' + f'{
q_val*1e6:.1f}' + r'\mu C$)')
143     ax_clean.legend(loc='upper right')
144
145     ax_clean.set_xlim(-limit, limit)
146     ax_clean.set_ylim(-limit, limit)
147     ax_clean.set_aspect('equal')
148     ax_clean.set_xlabel(r'$x$')
149     ax_clean.set_ylabel(r'$y$')
150
151     return fig_clean
152
153 fig = plt.figure(figsize=(9, 8))
154 plt.subplots_adjust(left=0.08, right=0.88, bottom=0.25)
155 ax = fig.add_subplot(111)
156
157 cbar_ax = fig.add_axes([0.92, 0.15, 0.02, 0.7])
158 cbar = None
159
160 def save_current(event):
161     mode = radio.value_selected
162     a_val = float(text_box_a.text)
163     q_val = float(text_box_q.text) * 1e-6
164     q_uc = float(text_box_q.text)
165     if mode == "Charge Mode":
166         fname = f"charge-{q_uc:.1f}-{a_val:.1f}.png"
167     elif mode == "Potential Mode":
168         fname = f"potential-{q_uc:.1f}-{a_val:.1f}.png"
169     else:
170         fname = f"field-{q_uc:.1f}-{a_val:.1f}.png"
171     fig_clean = create_plot(a_val, q_val, mode)
172     fig_clean.savefig(fname, dpi=150, bbox_inches='tight', pad_inches=0.1)
173     plt.close(fig_clean)
174     print(f"Saved {fname}")
175
176 def update(val):
177     global cbar
178     a_val = float(text_box_a.text)
179     q_val = float(text_box_q.text) * 1e-6
180     xs, ys, X, Y, pot, Ex, Ey, emag, sigma = get_physics_data_2d(R_curr, a_val,
q_val)
181
182     ax.clear()
183
184     mode = radio.value_selected
185
186     if cbar is not None:
187         cbar.ax.clear()
188         cbar = None
189
190     if mode == "Charge Mode":

```

```

191     vmin, vmax = np.min(sigma), np.max(sigma)
192
193     points = np.array([xs, ys]).T.reshape(-1, 1, 2)
194     segments = np.concatenate([points[:-1], points[1:]], axis=1)
195     lc = LineCollection(segments, cmap='viridis', norm=Normalize(vmin=vmin, vmax=
196     =vmax))
197     lc.set_array(sigma)
198     lc.set_linewidth(2)
199     ax.add_collection(lc)
200
201     sm = plt.cm.ScalarMappable(cmap='viridis', norm=Normalize(vmin=vmin, vmax=
202     vmax))
203     sm.set_array([])
204     cbar = plt.colorbar(sm, cax=cbar_ax)
205     cbar.set_label(r'$\sigma$ (C/m2)')
206
207     ax.set_title(r"Surface Charge Density ($a=" + f"{a_val:.2f}$)")
208
209 elif mode == "Potential Mode":
210     pot_clim = (100, 50000)
211     levels = np.linspace(pot_clim[0], pot_clim[1], 25)
212
213     pot_plot = pot.copy()
214     inside = np.sqrt(X**2 + Y**2) < R_curr
215     pot_plot[inside] = np.nan
216
217     contour = ax.contourf(X, Y, pot_plot, levels=levels, cmap='Reds')
218     ax.contour(X, Y, pot_plot, levels=levels, colors='darkred', linewidths=0.5)
219
220     cbar = fig.colorbar(contour, cax=cbar_ax)
221     cbar.set_label(r'Potential (V)')
222
223     circle = Circle((0, 0), R_curr, fill=True, facecolor='white', edgecolor='
224     black', linewidth=2)
225     ax.add_patch(circle)
226     ax.set_title(r"Potential Field ($a=" + f"{a_val:.2f}$)")
227
228 elif mode == "Field Mode":
229     skip = 6
230
231     Xs = X[::skip, ::skip]
232     Ys = Y[::skip, ::skip]
233     Exs = Ex[::skip, ::skip]
234     Eys = Ey[::skip, ::skip]
235     emag_s = emag[::skip, ::skip]
236
237     dist_from_sphere = np.sqrt(Xs**2 + Ys**2)
238     dist_from_charge = np.sqrt((Xs-a_val)**2 + Ys**2)
239     mask = (dist_from_sphere > R_curr + 0.2) & (dist_from_charge > 0.3)
240
241     Xm = Xs[mask]
242     Ym = Ys[mask]

```

```

240     Exm = Exs[mask]
241     Eym = Eys[mask]
242     emag_m = emag_s[mask]
243
244     emag_nonzero = emag_m[emag_m > 0]
245     if len(emag_nonzero) > 0:
246         emag_min = np.min(emag_nonzero)
247         emag_max = np.max(emag_nonzero)
248         use_log = True
249     else:
250         emag_min, emag_max = 1, 100
251         use_log = False
252
253     lens = np.sqrt(Exm**2 + Eym**2)
254     Ex_unit = np.zeros_like(Exm)
255     Ey_unit = np.zeros_like(Eym)
256     nonzero = lens > 0
257     Ex_unit[nonzero] = Exm[nonzero] / lens[nonzero]
258     Ey_unit[nonzero] = Eym[nonzero] / lens[nonzero]
259
260     if use_log:
261         quiver = ax.quiver(Xm, Ym, Ex_unit * 1.5, Ey_unit * 1.5,
262                             emag_m, cmap='plasma',
263                             norm=LogNorm(vmin=emag_min, vmax=emag_max),
264                             pivot='mid', width=0.003, headwidth=3, headlength=3,
headaxislength=3)
265         sm = plt.cm.ScalarMappable(cmap='plasma', norm=LogNorm(vmin=emag_min,
vmax=emag_max))
266     else:
267         quiver = ax.quiver(Xm, Ym, Ex_unit * 1.5, Ey_unit * 1.5,
268                             emag_m, cmap='plasma',
269                             pivot='mid', width=0.003, headwidth=3, headlength=3,
headaxislength=3)
270         sm = plt.cm.ScalarMappable(cmap='plasma')
271
272     sm.set_array([])
273     cbar = plt.colorbar(sm, cax=cbar_ax)
274     cbar.set_label(r'$|E|$ (V/m)')
275
276     circle = Circle((0, 0), R_curr, fill=False, color='white', linewidth=2)
277     ax.add_patch(circle)
278     ax.set_title(r"Electric Field ($a=" + f"{a_val:.2f}$)")
279
280     circle = Circle((0, 0), R_curr, fill=False, color='cornflowerblue', linewidth=2)
281     ax.add_patch(circle)
282     ax.plot(a_val, 0, 'r.', markersize=8, label=r'Source Charge ($q=' + f'{q_val*1e6}
:.1f)' + r'\mu C$)')
283     ax.legend(loc='upper right')
284
285     ax.set_xlim(-limit, limit)
286     ax.set_ylim(-limit, limit)
287     ax.set_aspect('equal')

```

```

288     ax.set_xlabel(r'$x$')
289     ax.set_ylabel(r'$y$')
290
291     fig.canvas.draw_idle()
292
293 ax_text_a = plt.axes((0.15, 0.04, 0.08, 0.04))
294 text_box_a = TextBox(ax_text_a, r'$a$ = ', initial='4.0')
295 text_box_a.on_text_change(update)
296
297 ax_text_q = plt.axes((0.28, 0.04, 0.08, 0.04))
298 text_box_q = TextBox(ax_text_q, r'$q$ (\mu C)$ = ', initial='10.0')
299 text_box_q.on_text_change(update)
300
301 ax_radio = plt.axes((0.02, 0.6, 0.12, 0.2), frameon=False)
302 radio = RadioButtons(ax_radio, ('Charge Mode', 'Potential Mode', 'Field Mode'))
303 radio.on_clicked(update)
304
305 ax_save = plt.axes((0.8, 0.04, 0.1, 0.04))
306 btn_save = Button(ax_save, 'Save')
307 btn_save.on_clicked(save_current)
308
309 update(4.0)
310
311 radio.set_active(0)
312 update(None)
313 q_val = 10e-6
314 q_uc = 10.0
315 a_val = 4.0
316 fig_clean = create_plot(a_val, q_val, 'Charge Mode')
317 fig_clean.savefig(f"charge-{q_uc}-{a_val}.png", dpi=150, bbox_inches='tight',
318                 pad_inches=0.1)
319 plt.close(fig_clean)
320 print(f"Saved charge-{q_uc}-{a_val}.png")
321
322 radio.set_active(1)
323 update(None)
324 fig_clean = create_plot(a_val, q_val, 'Potential Mode')
325 fig_clean.savefig(f"potential-{q_uc}-{a_val}.png", dpi=150, bbox_inches='tight',
326                 pad_inches=0.1)
327 plt.close(fig_clean)
328 print(f"Saved potential-{q_uc}-{a_val}.png")
329
330 radio.set_active(2)
331 update(None)
332 fig_clean = create_plot(a_val, q_val, 'Field Mode')
333 fig_clean.savefig(f"field-{q_uc}-{a_val}.png", dpi=150, bbox_inches='tight',
334                 pad_inches=0.1)
335 plt.close(fig_clean)
336 print(f"Saved field-{q_uc}-{a_val}.png")
337
338 radio.set_active(0)
339 update(None)

```

```
337  
338 plt.show()
```

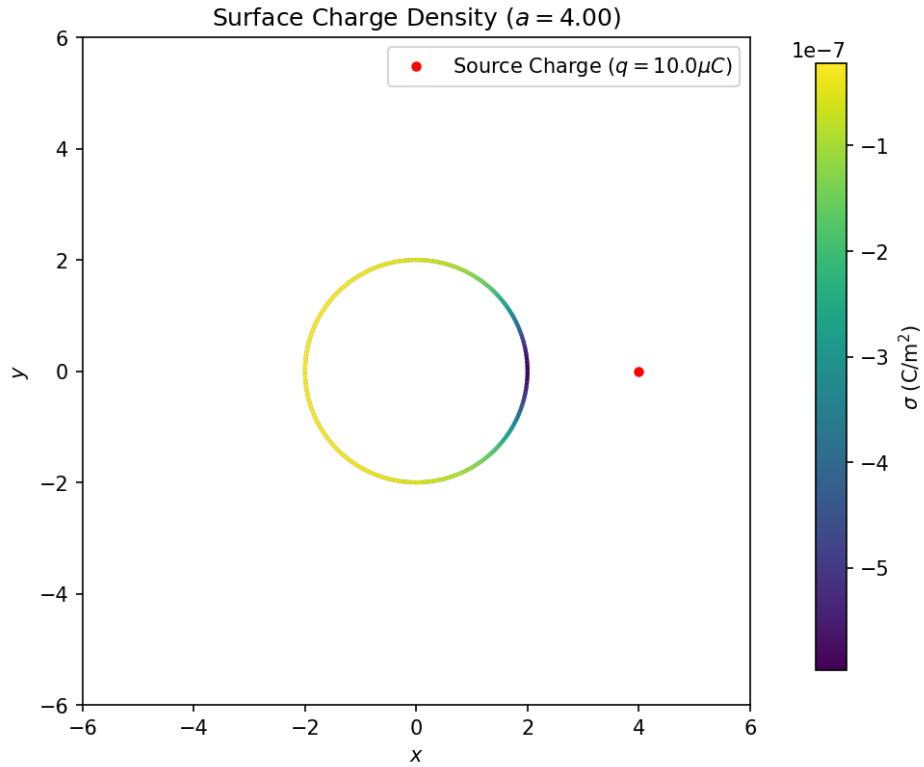


Figure 5: obtained surface charge density

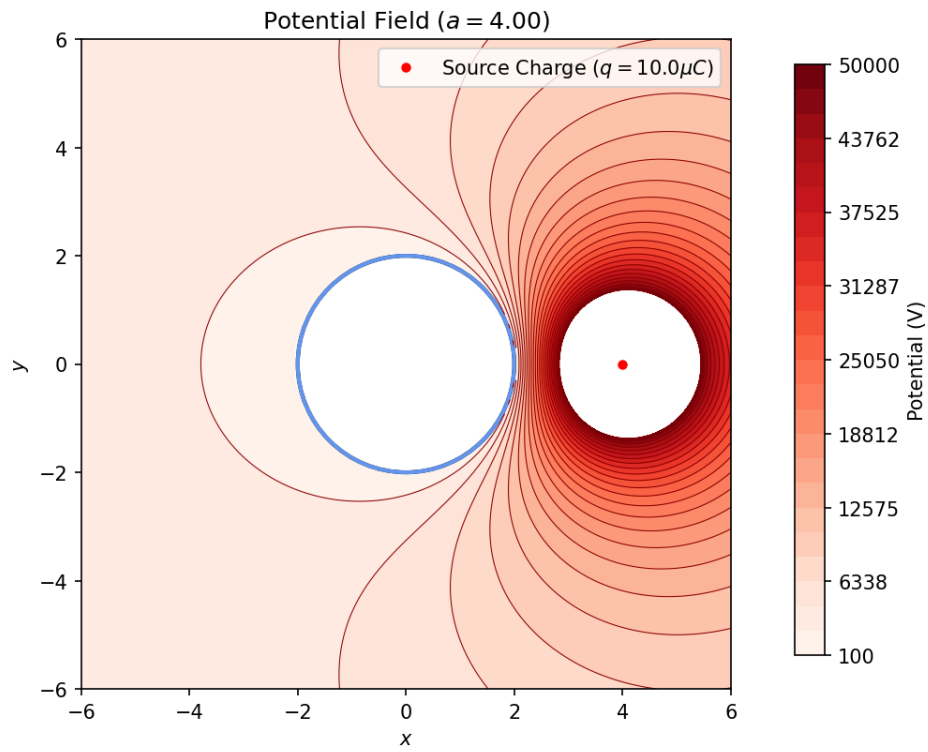


Figure 6: obtained potential contours

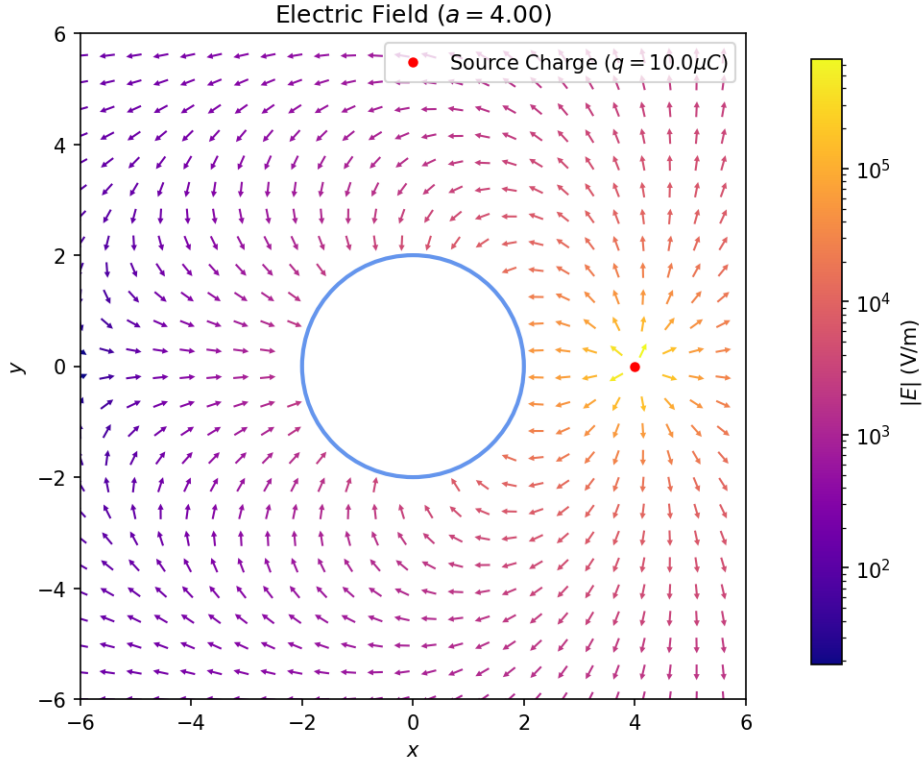


Figure 7: obtained electric field

3. Simulation of Electric Field Enhancement at a Sharp Conductor (Lightning Rod)

Simulate the electrostatic field distribution produced by a sharp conducting needle placed between two conducting planes representing ground and a charged cloud.

- Use a 100×100 computational grid.
- **Bottom boundary (ground):** Set the entire bottom row to $V = 0$.
- **Top boundary (cloud):** Set the entire top row to $V = 100$.
- **Needle conductor:** At the center of the grid, create a vertical conducting line extending from the ground up to the middle of the grid and fix it at $V = 0$.
- Solve Laplace's equation iteratively until the potential distribution converges.

Required outputs

- Generate a heat map (color map) of the potential distribution.
- Overlay the electric field vectors computed using $\mathbf{E} = -\nabla V$.
- Describe the electric field behavior near the tip of the needle and explain the physical reason for this effect.

Solution

TA Credit-Vedhamsh - python code for Lightning rod simulation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 100
5 V = np.zeros((N, N))
6
7 # Defining Boundaries
8 V[-1, :] = 100 # The Sky (Top)
9 V[0, :] = 0    # The Ground (Bottom)
10
11 # The lightning rod
12 needle_x = 50
13 needle_height = 40
14 V[0:needle_height, needle_x] = 0
15
16 is_fixed = np.zeros((N, N), dtype=bool)
17 is_fixed[-1, :] = True
18 is_fixed[0, :] = True
19 is_fixed[0:needle_height, needle_x] = True
20
21 # Averaging Loop
22 for i in range(10000):
23     V_old = V.copy()
24     V[1:-1, 1:-1] = 0.25 * (V_old[0:-2, 1:-1] + V_old[2:, 1:-1] +
25                             V_old[1:-1, 0:-2] + V_old[1:-1, 2:])
26     V[is_fixed] = V_old[is_fixed]
27
28
29 Ey, Ex = np.gradient(-V) # Field Calculation
30
31 plt.figure(figsize=(8, 8))
32
33 # Voltage Heat Map
34 plt.imshow(V, cmap='viridis', origin='lower')
35 plt.colorbar(label='Potential (V)')
36
37 plt.plot([needle_x, needle_x], [0, needle_height], color='white', linewidth=3, label
38         ="Lightning Rod")
39
40 skip = 5
41 y, x = np.mgrid[0:N:skip, 0:N:skip]
42 plt.quiver(x, y, Ex[::skip, ::skip], Ey[::skip, ::skip], color='white', alpha=0.5)
43
44 plt.title("Lightning Rod Simulation")
45 plt.legend()
46 plt.show()
```

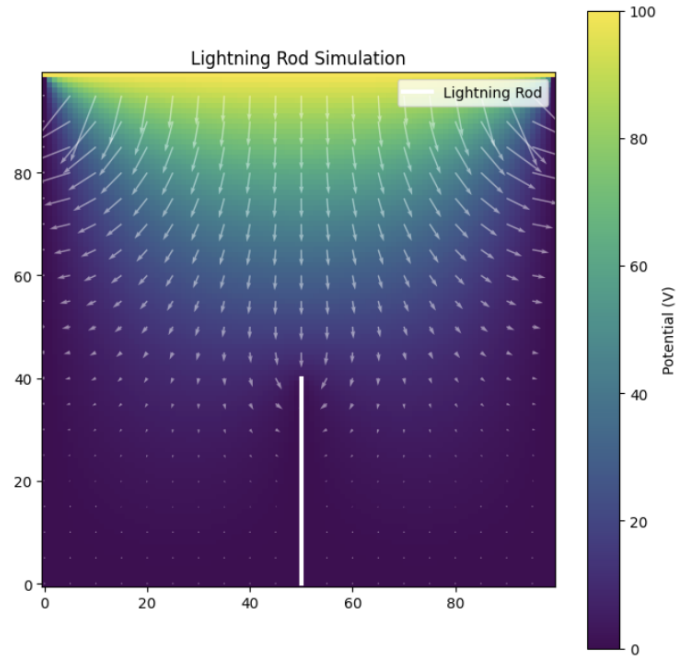


Figure 8: obtained simulation plot for the lightning rod
